PS03-AI · THỰC HÀNH TỪNG BƯỚC

Từ bảng dữ liệu đến hình biết nói

Hướng dẫn pandas, plotnine và notebook VS Code

Nguyễn Bích Vân · IAI-UET-VNU

Mã mẫu dùng một bộ dữ liệu riêng để học cú pháp. Hãy thay tên cột, biên khoảng và cách chọn mẫu số theo câu hỏi của từng bài; không sao chép một đoạn mã rồi coi kết quả là đáp án cho mọi bài.

1. Mở đúng thư mục, chọn đúng kernel

Giải nén toàn bộ gói rồi mở **File → Open Folder** trong VS Code. Chọn thư mục chứa `data/`, không chỉ mở riêng một notebook từ trong ZIP. Cài hai tiện ích **Python** và **Jupyter** của Microsoft nếu máy chưa có.

Mở `00_Learn_Plotnine.ipynb`. Ở góc trên bên phải, bấm **Select Kernel** và chọn Python muốn dùng. Chạy ô đầu tiên để biết kernel đang dùng chương trình nào:

```
import sys
print(sys.executable)
```

Nếu thiếu thư viện, tạo một ô Code và chạy:

```
%pip install -r requirements.txt
```

Sau khi cài, chọn **Restart Kernel** rồi chạy từ đầu. Lệnh `%pip` này dành cho **ô notebook**, không dán vào PowerShell. Nó tránh cài thư viện vào Python khác với kernel đang chạy. Máy có cả Anaconda và Python 3.13 cần đặc biệt chú ý đường dẫn ở ô kiểm tra.

Chạy một ô: bấm nút tam giác cạnh ô hoặc nhấn Shift+Enter. **Chạy cả notebook:** chọn Run All. Kết quả hiện *ngay dưới ô mã*, không phải Terminal. Trong PowerShell, Shift+Enter có thể xuống dòng chờ nhập; dấu nhắc `PS ...>` và » không thuộc mã Python và không được sao chép vào ô.

Các notebook trong gói gồm:

Tệp	Dùng để làm gì
`00_Learn_Plotnine.ipynb`	Chạy mẫu, đọc giải thích và thử thay một lệnh.
`PS03_AI_Student.ipynb`	Làm Bài 6–10 ở các ô TODO; ghi nhận xét.

Chưa cần viết nhiều mã. Trước hết hãy chạy thành công notebook mẫu; sau đó quay lại từng bài để chọn cột, biên khoảng và mẫu số phù hợp. Gói đã có dữ liệu local, không cần mạng để đọc CSV sau khi cài thư viện. Hướng dẫn thao tác notebook tham khảo [5–6].

2. Đọc bảng trước khi vẽ

Bộ dữ liệu mẫu `demo_drink_orders.csv` mô tả 24 lượt nhận đồ uống sau một buổi seminar AI. Đây là **ví dụ giả lập học cú pháp**, không phải dữ liệu Bài 6–10. Mỗi hàng là một đơn; `order_type` cho biết mang đi hay tại chỗ, `wait_time_min` là số phút chờ, `item_count` là số món và `shift` là sáng/chiều.

```
from pathlib import Path
import numpy as np
import pandas as pd
from IPython.display import display
from plotnine import (
    ggplot, aes, geom_point, geom_histogram, after_stat,
    geom_rect, geom_bar, geom_col, geom_hline,
    geom_boxplot, geom_line, facet_wrap, labs, scale_x_continuous
)

ROOT = Path.cwd()
DATA = ROOT / "data"
demo = pd.read_csv(DATA / "demo_drink_orders.csv")
display(demo.head())
print("The number of rows, the number of columns:", demo.shape)
print(demo.dtypes)
print(demo.isna().sum())
```

`Path.cwd()` là thư mục làm việc hiện tại. Notebook không có biến `__file__`. Nếu báo không tìm thấy CSV, in `DATA.resolve()` rồi kiểm tra có mở đúng thư mục không; đừng sửa số liệu hoặc tên cột để né lỗi đường dẫn.

`head()` cho vài dòng đầu, `shape` cho số hàng/cột, `dtypes` cho kiểu dữ liệu, `isna().sum()` đếm ô thiếu theo cột. Cột số có ô thiếu có thể được đọc thành kiểu số thực dù các giá trị quan sát là số nguyên.

Khi phân tích hai cột, chỉ bỏ giá trị thiếu ở các cột thực sự cần dùng:

```
x = demo.dropna(subset=["order_type", "wait_time_min"]).copy()
print("The number of avoided rows:", len(demo) - len(x))
```

Không thay thời gian thiếu bằng 0: thiếu thông tin khác với chờ 0 phút.

3. Một biểu đồ được xây bằng các lớp

Việc vẽ biểu đồ bắt đầu bằng ba quyết định: bảng nào, biến nào đặt ở đâu, dùng hình gì để biểu diễn. Ví dụ sau vẽ mỗi đơn đồ uống thành một điểm:

```
p = (
    ggplot(demo, aes(x="item_count", y="wait_time_min", color="order_type"))
    + geom_point(size=2, alpha=0.7)
    + labs(x="The number of items in the order", y="Waiting time (mins)",
           color="Order type", title="Each order is a point")
)
display(p)
```

`ggplot(demo,...)` chọn bảng; `aes` ánh xạ số món, thời gian chờ và kiểu đơn tới vị trí/màu; `geom_point` vẽ điểm. `labs` đổi nhãn hiển thị, *không đổi tên cột*. Hai tham số `size` và `alpha` là thiết lập hiển thị. Khi biến nằm trong `aes(color="order_type")`, màu mang thông tin phân nhóm. Tên biến phải đặt trong dấu nháy và khớp CSV.

Dấu cộng nối các lớp; dấu ngoặc ngoài cho phép viết biểu thức trên nhiều dòng. Trong notebook có thể để `p` ở dòng cuối, hoặc dùng `display(p)`; trong vòng lặp nên gọi `display(p)` cho từng hình. Chỉ viết `print(p)` không phải cách chắc chắn để hiện hình.

Sau khi vẽ, lưu hình để nộp:

```
OUT = ROOT / "example_figures"
OUT.mkdir(exist_ok=True)
p.save(OUT / "scatter.png", width=7, height=4, dpi=150)
p.save(OUT / "scatter.pdf", width=7, height=4)
```

PNG dễ chèn báo cáo; PDF giữ đồ họa vector. Các lệnh hiển thị và lưu theo hướng dẫn chính thức [1]. Các hình trong gói không cần đặt bảng màu cố định để làm bài.

Thử thay một thứ: bỏ ánh xạ màu, giữ các thành phần khác. Những điểm nào vẫn còn và thông tin nào không còn được hiển thị? Đây là cách học nhanh hơn sao chép cả đoạn mà không đọc.

4. Lọc, tạo biến và tổng hợp có chủ đích

Ba thao tác sau tác động lên bảng theo ba cách khác nhau:

```
chon = demo.query("item_count >= 2 and wait_time_min < 8").copy()
chon["long_wait"] = chon["wait_time_min"] >= 6
bang = (
    chon.groupby("order_type", observed=True)
    .agg(n=("id", "size"),
         n_long_wait=("long_wait", "sum"),
         long_wait_rate=("long_wait", "mean"))
    .reset_index()
)
bang["percent"] = 100 * bang["long_wait_rate"]
display(bang)
```

`query` giữ từng hàng thỏa đồng thời hai điều kiện. Dòng gán tạo một biến logic. `groupby` chia các hàng đã giữ theo kiểu đơn; `agg` tóm tắt từng nhóm, nên mỗi hàng kết quả lúc này là *một nhóm*, không còn là một đơn.

`size` đếm số hàng; `count` đếm số giá trị không thiếu của cột được chọn. Với dữ liệu đã bỏ thiếu phù hợp, lấy trung bình biến logic là tỉ lệ đúng. Nếu dùng trực tiếp phép so sánh trên cột có NaN, NaN có thể trở thành False và làm sai mẫu số; vì vậy phải xác định chính sách thiếu *trước* khi tạo biến chỉ báo.

Cách lọc tương đương bằng mặt nạ:

```
chon = demo.loc[
    (demo["item_count"] >= 2) & (demo["wait_time_min"] < 8)
].copy()
```

Trong `query` dùng `and/or`; với hai Series dùng `&/|` và đặt ngoặc quanh mỗi điều kiện. Còn `demo[["order_type","wait_time_min"]]` là *chọn cột*, không phải lọc hàng.

Tự đọc lại bằng lời: “Trong các đơn có ít nhất hai món và chờ dưới tám phút, mỗi kiểu đơn có bao nhiêu đơn và bao nhiêu phần chờ ít nhất sáu phút?” Nếu câu này không phải câu hỏi muốn nghiên cứu, cần sửa thao tác trước khi vẽ.

5. Histogram: biên khoảng và chuẩn hóa

Bản số đếm cho biết số quan sát trong mỗi khoảng. Đặt biên rõ ràng thay vì để phần mềm tự chọn:

```
edges = [0, 2, 4, 6, 8, 10, 12]
p_count = (
    ggplot(demo, aes(x="wait_time_min"))
    + geom_histogram(breaks=edges, closed="left")
    + labs(x="Waiting time (mins)", y="The number of orders")
)
display(p_count)
```

`closed="left"` dùng khoảng lấy đầu trái. Dữ liệu mẫu lớn nhất là 10, nhỏ hơn biên cuối 12. Luôn chọn biên phủ tất cả quan sát cần phân tích; nếu lọc bớt, phải nói rõ tập còn lại. Với NumPy, khoảng cuối của `histogram` có tính đầu phải; các ví dụ Python của bộ này chọn biên cuối lớn hơn max nên khác biệt đó không làm đổi số đếm.

Để diện tích của mỗi histogram là 100%, dùng mật độ phần trăm:

```
p_density = (
    ggplot(demo, aes(x="wait_time_min", y=after_stat("100*density")))
    + geom_histogram(breaks=edges, closed="left")
    + facet_wrap("~ order_type", scales="fixed")
    + labs(x="Waiting time (mins)", y="Density (% / min)")
)
display(p_density)
```

`after_stat` lấy kết quả sau phép đếm của lớp histogram. Trong từng ô phân nhỏ, mật độ được chuẩn hóa theo dữ liệu của ô; giữ `scales="fixed"` để các trục so sánh trực tiếp. Không dùng `ncount` thay mật độ: đưa cột cao nhất về 1 không đồng nghĩa tổng diện tích bằng 1 [2–3].

Công thức để đối chiếu: nếu n_j quan sát nằm trong khoảng rộng Δ_j, trên n quan sát hợp lệ, thì $h_j = 100 n_j/(n\Delta_j)$. Chiều cao mang đơn vị %/đơn vị trục ngang. $100 n_j/n$ là phần trăm *cả khoảng*, không phải mật độ khi $\Delta_j \neq 1$.

6. Tự tính mật độ để nhìn rõ mẫu số

Với khoảng không đều, có thể lập bảng bằng NumPy rồi vẽ các hình chữ nhật bằng `geom_rect`. Cách này giúp kiểm tra độc lập với thống kê tự động của plotnine:

```
edges = np.array([0, 3, 5, 9, 12])
values = demo["wait_time_min"].dropna().to_numpy()
counts, _ = np.histogram(values, bins=edges)
assert counts.sum() == len(values)

bins = pd.DataFrame({
    "bin_left": edges[:-1], "bin_right": edges[1:], "n": counts
})
bins["percent"] = 100 * bins["n"] / len(values)
bins["percent_density"] = bins["percent"] / (bins["bin_right"] - bins["bin_left"])
display(bins)
print("Total area:",
      (bins["percent_density"] * (bins["bin_right"] - bins["bin_left"])).sum())

p = (
    ggplot(bins, aes(xmin="bin_left", xmax="bin_right",
                     ymin=0, ymax="percent_density"))
    + geom_rect()
    + scale_x_continuous(breaks=edges.tolist())
    + labs(x="Waiting time (mins)", y="Density (% / phut)")
)
display(p)
```

`edges[:-1]` bỏ biên cuối để lấy mép trái; `edges[1:]` bỏ biên đầu để lấy mép phải. `np.diff(edges)` cũng cho độ rộng các khoảng. Mỗi hàng của `bins` là một khoảng; `geom_rect` vẽ nó bằng hai mép trên trục x và hai mép trên trục y.

Đối với nhiều nhóm, thực hiện bảng này *riêng trong từng nhóm*. Mẫu số phải là `len(values)` của nhóm đang xét, không phải tổng toàn bộ. Kiểm tra $\sum h_j \Delta_j$ cho từng nhóm, không chỉ kiểm tra tổng gộp.

Cách bắt lỗi nhanh: nếu diện tích ra 60% hoặc 150%, hãy kiểm tra số quan sát nằm ngoài biên, quy tắc đầu mút và mẫu số trước khi chỉnh màu hay độ rộng hình.

7. Biểu đồ cột: có bảng tần số rồi thì dùng geom_col

`geom_bar()` tự đếm từ từng quan sát; `geom_col()` lấy chiều cao từ một cột số đã tính [4]. Với cột tỉ lệ, dùng `geom_col`, không đếm lại số hàng của bảng tóm tắt.

```
bang = (
    demo.groupby("order_type", observed=True)
    .agg(n=("id", "size"),
         n_long_wait=("wait_time_min", lambda s: (s >= 6).sum()))
    .reset_index()
)
bang["percent"] = 100 * bang["n_long_wait"] / bang["n"]
p = (ggplot(bang, aes(x="order_type", y="percent"))
     + geom_col()
     + labs(x="Order type", y="Waiting rate of at least 6 mins (%)"))
display(p)
```

Giải thích một ô bằng lời: mẫu số là tổng số đơn của kiểu đó; tử số là số đơn cùng kiểu chờ từ 6 phút trở lên. Đừng lọc chỉ các đơn chờ lâu rồi mới tính tổng đơn.

Để giữ đủ các mức rời rạc kể cả mức không xuất hiện, dùng `reindex`:

```
digit = demo["wait_time_min"].astype(int) % 10
freq = digit.value_counts().reindex(range(10), fill_value=0)
bang_digit = pd.DataFrame({"last_digit": range(10), "n": freq.to_numpy()})
bang_digit["percent"] = 100 * bang_digit["n"] / len(digit)
p = (ggplot(bang_digit, aes(x="last_digit", y="percent"))
     + geom_col() + geom_hline(yintercept=10, linetype="dashed")
     + scale_x_continuous(breaks=list(range(10))))
display(p)
```

Mẫu 24 đơn chỉ để học cú pháp; chưa phải minh chứng về thói quen ghi số. Trước khi đổi sang số nguyên phải kiểm tra dữ liệu thật có phải số nguyên không, không lặng lẽ cắt phần thập phân.

8. Chia nhóm số, chuyển dạng dài, rồi tách ô

`pd.cut` biến giá trị số thành nhãn khoảng. Cần dùng đúng cùng một quy tắc cho tử số và mẫu số:

```
x = demo.copy()
x["wait_bin"] = pd.cut(
    x["wait_time_min"], bins=[0, 3, 6, 9, 12], right=False
)
assert x["wait_bin"].notna().all()
bang = (x.groupby("wait_bin", observed=False)
        .agg(n=("id", "size")).reset_index())
bang["wait_bin"] = bang["wait_bin"].astype(str)
display(bang)
```

`right=False` chọn khoảng $[a, b)$. `observed=False` có thể giữ cả các mức khoảng trống đã định nghĩa. Nếu tính tỉ lệ, dùng `n.replace(0,np.nan)` làm mẫu số để nhóm rỗng cho NA, không phải 0%.

Khi có nhiều cột đo cùng đại lượng, chuyển bảng rộng sang dài. Ví dụ độc lập:

```
thu = pd.DataFrame({
    "id": [1, 2, 3], "trial_1": [4.2, 5.1, 3.8],
    "trial_2": [3.9, 5.4, 4.0], "trial_3": [4.0, 5.0, 3.6]
})
dai = thu.melt(id_vars="id", value_vars=["trial_1", "trial_2", "trial_3"],
               var_name="trial", value_name="time_s")
display(dai)
p = (ggplot(dai, aes(x="trial", y="time_s", group="id"))
     + geom_point() + geom_line()
     + labs(x="Measurement", y="Time (sec)"))
display(p)
```

Mỗi hàng mới là một cặp ID–lần đo. Dòng `group="id"` cho phép nối các lần đo của *cùng đơn vị*; không nối các quan sát độc lập chỉ vì chúng đứng cạnh nhau trong bảng. Trong Bài 8, dùng ý tưởng `melt` cho ba điểm thành phần rồi dùng `facet_wrap`; không cần đường nối.

9. Từ mã mẫu đến bài nộp của bạn

Mỗi bài Python nên có ba phần: **câu hỏi đang trả lời**, **mã và kết quả**, **nhận xét**. Không để người đọc phải suy đoán ý nghĩa của bảng hoặc ảnh.

Bài	Nên bắt đầu từ mẫu nào?	Phần bạn phải tự quyết định/viết
6	Histogram + chia ô (mục 5–6)	Bảng theo tác vụ; kiểm tra diện tích; so sánh hai phân phối.
7	Biểu đồ cột + reindex (mục 7)	Tỉ lệ chữ số theo cách ghi; giải thích cơ chế có thể xảy ra.
8	Histogram + melt (mục 5,8)	So sánh biên 5/1 điểm; kiểm tra các đợt và thành phần.
9	groupby + boxplot	Trung bình gộp/nhóm; tính trọng số; diễn giải sự đảo chiều.
10	cut + tử số/mẫu số (mục 7–8)	Không lọc mất mẫu số; so sánh số lỗi và tỉ lệ lỗi.

Danh mục kiểm tra trước khi nộp

1. Restart Kernel, chạy Run All từ đầu. Không dựa vào biến còn sót sau một lần chạy cũ.
2. Mọi CSV được đọc từ `data/`; ghi rõ bao nhiêu hàng bị bỏ và lý do.
3. Mọi biểu đồ có tên biến, đơn vị, biên chia khi cần; hình so sánh dùng thang đo nhất quán.
4. Tỉ lệ luôn có tử số và mẫu số giải thích được. Nếu $n = 0$, để NA và ghi lý do.
5. Không thay dữ liệu để làm đẹp một kết luận. Các câu nhận xét dựa trên hình/bảng vừa tạo, không chép câu mẫu.
6. Phân biệt mô tả dữ liệu giả lập, dự đoán theo giả thiết và khẳng định nguyên nhân.

Lỗi thường gặp. `ModuleNotFoundError`: cài bằng `%pip` trong đúng kernel. `NameError`: chạy lại ô nạp dữ liệu/nhập thư viện. `KeyError`: xem `df.columns.tolist()` và đối chiếu tên cột. Không thấy hình: dùng `display(p)`. Tỉ lệ sai: kiểm tra lọc thiếu, nhóm rỗng và mẫu số. Chỉ có chữ TODO không phải lỗi chương trình: đó là phần bài sinh viên cần bổ sung.

Tài liệu chính thức để tra cú pháp

Các nguồn dưới đây chỉ dùng cho cú pháp phần mềm; dữ liệu trong bộ bài tập là giả lập mới. Mã đi kèm đã kiểm tra với Python 3.13.5, NumPy 2.3.5, pandas 2.2.3 và plotnine 0.15.3. Giao diện máy khác có thể khác nhẹ; xem phiên bản thực tế trong ô đầu notebook.

1. plotnine, hiển thị và lưu hình: https://plotnine.org/guide/export.html.
2. plotnine, histogram và các thang biểu diễn: https://plotnine.org/reference/geom_histogram.html.
3. plotnine, chia ô: https://plotnine.org/reference/facet_wrap.html.
4. plotnine, cột từ dữ liệu đã tổng hợp: https://plotnine.org/reference/geom_col.html.
5. VS Code, sử dụng notebook: https://code.visualstudio.com/docs/datascience/jupyter-notebooks.
6. IPython, lệnh `%pip`: https://ipython.readthedocs.io/en/stable/interactive/magics.html#magic-pip.

Không cần tra thêm tài liệu để làm các phép tính cốt lõi trong phiếu bài tập này. Hãy dùng phần mềm để kiểm chứng câu trả lời, không để phần mềm quyết định câu hỏi thay mình.